📦 MÓDULO JAVASCRIPT · INICIACIÓN PRÁCTICA

Objetos Literales en JavaScript

Guía didáctica fundamental para principiantes: de variables sueltas a estructuras organizadas, pares clave-valor, lectura por punto y corchetes, mutación de propiedades, métodos sencillos con `this` y colecciones con arrays de objetos.

🧱 1. Concepto y Sintaxis

Aprende por qué los objetos resuelven el caos de las variables sueltas y cómo crearlos con la notación literal `{}`.

🔍 2. Lectura y Escritura

Domina el acceso con punto (`objeto.prop`), con corchetes (`objeto['prop']`) y cómo modificar o añadir claves.

⚙️ 3. Métodos y 'this'

Añade funciones dentro de tus objetos y comprende de forma sencilla cómo `this` apunta al propio objeto.

📋 4. Arrays de Objetos

Modela listas reales del mundo web (catálogos de productos, listas de alumnos) y recórrelas fácilmente con `for ... of`.

Ramón Abramo

Profesor de Desarrollo Web y Programación Frontend · Santander (Cantabria)

Certificado de Profesionalidad IFCD0110

Edición 2026 · Documento Oficial con Marcadores

📑 ÍNDICE GENERAL DE CONTENIDOS

(Haz clic en cualquier sección para navegar)

📦 Portada Oficial y Presentación	Pág. 1	1. ¿Por qué necesitamos objetos en JS?	Pág. 2
2. Anatomía de un objeto literal ({})	Pág. 2	3. Lectura de propiedades: Punto vs Corchetes	Pág. 3
4. Modificar, añadir y borrar (delete)	Pág. 4	5. Métodos sencillos y la palabra 'this'	Pág. 5
6. Arrays de objetos: Colecciones y for...of	Pág. 6	7. Mostrar objetos en el DOM y Buenas Prácticas	Pág. 7

📦 1. ¿Por qué necesitamos objetos en JavaScript?

Hasta ahora, para guardar información sobre una entidad (por ejemplo, un alumno) usábamos variables sueltas: `let nombreAlumno = "Ana"; let edadAlumno = 20; let cursoAlumno = "Web";`. Esto plantea graves problemas en proyectos reales:

ENFOQUE	CÓDIGO DE EJEMPLO	PROBLEMA / VENTAJA

❌ **Variables Sueltas**	`let nombre = "Ana";`
	`let edad = 20;`
	`let aprobado = true;`
		Si tenemos 30 alumnos, necesitamos 90 variables independientes. No están vinculadas entre sí y el código se vuelve ingobernable.

✅ **Objeto Literal**	`const alumno = {`
	`  nombre: "Ana",`
	`  edad: 20,`
	`  aprobado: true`
	`};`
		Todos los datos de la entidad viajan juntos en una sola variable organizada. Es limpio, intuitivo y fácil de mantener.

🧱 2. Anatomía de un Objeto Literal ({})

En JavaScript, un **objeto literal** se declara abriendo y cerrando llaves `{}`. Dentro colocamos pares formados por un **nombre de propiedad (clave)** y su **valor**, separados por dos puntos `:`. Cada par se separa del siguiente mediante una coma `,`.

📝 Reglas sintácticas básicas:

- **Clave (key):** Nombre identificador de la propiedad (ej. `nombre`, `precio`, `stock`).
- **Dos puntos (:):** Separan la clave del valor. ¡Nunca uses `=` dentro del objeto!
- **Valor (value):** Puede ser cualquier tipo de dato de JS: texto, número, booleano, array u otra función.
- **Coma (,):** Separa cada propiedad. La última propiedad no necesita coma obligatoria.

```
// Objeto sencillo que modela un libro
const libro = {
  titulo: "El Quijote",
  autor: "Miguel de Cervantes",
  paginas: 863,
  prestado: false
};
```

> 💡 **Consejo del Profesor Ramón:**
> Declara tus objetos con `const`. En JavaScript, `const` impide que reasignes la variable a otro valor (por ejemplo `libro = "otro"`), pero **SÍ te permite modificar, añadir o borrar sus propiedades internas**.

🔍 3. Lectura de Propiedades: Notación de Punto vs Corchetes

Para consultar o leer el valor guardado en una propiedad disponemos de dos formas oficiales. Ambas son totalmente válidas, pero tienen casos de uso específicos:

👉 1. Notación de Punto (La más habitual)

Escribimos el nombre del objeto, un punto `.` y el nombre de la propiedad. Es la más limpia y la que más usarás:

```
const persona = { nombre: "Carlos", edad: 28 };

console.log(persona.nombre); // "Carlos"
console.log(persona.edad);   // 28
```

📦 2. Notación de Corchetes (Dinámica)

Escribimos el nombre del objeto y entre corchetes `['clave']` la propiedad como una cadena de texto:

```
console.log(persona['nombre']); // "Carlos"
console.log(persona['edad']);   // 28
```

¿Cuándo es OBLIGATORIO usar la notación de corchetes?

En el 90% de los casos usarás el punto, pero existen **dos situaciones imprescindibles** donde el punto falla y los corchetes son la única solución:

SITUACIÓN	¿POR QUÉ FALLA EL PUNTO?	SOLUCIÓN CON CORCHETES

1. El nombre viene de una variable	`let campo = "nombre";`
	`persona.campo; // undefined!`
	Busca literalmente la clave llamada "campo".
		`let campo = "nombre";`
		`persona[campo]; // "Carlos"`
		Evalúa el valor de la variable.

2. La clave contiene espacios o guiones	`usuario.codigo postal; // Error sintaxis!`
	JavaScript no permite espacios tras el punto.
		`usuario["codigo postal"];`
		Entre comillas se permite cualquier carácter.

❓ ¿Qué ocurre si leemos una propiedad que no existe?

A diferencia de otros lenguajes de programación, JavaScript **no da un error fatal** si intentas leer una clave inexistente: simplemente devuelve `undefined`.

```
const coche = { marca: "Seat", modelo: "Ibiza" };

console.log(coche.color); // Devuelve: undefined (no da error)

// ⚠️ ¡CUIDADO! Intentar leer algo de undefined sí provoca un error fatal:
// console.log(coche.color.toUpperCase()); // TypeError: Cannot read properties of undefined
```

> ⚠️ **Error clásico de principiante:**
> Al usar corchetes con una clave literal, **no olvides las comillas**: `alumno['nombre']`. Si escribes `alumno[nombre]` sin comillas, JavaScript creerá que `nombre` es una variable y si no existe provocará un `ReferenceError: nombre is not defined`.

🖍️ 4. Modificar, Añadir y Borrar Propiedades (CRUD Básico)

Los objetos en JavaScript son estructuras vivas y dinámicas en memoria. Puedes cambiar sus valores, añadir nuevas propiedades o eliminar las existentes en cualquier momento durante la ejecución de la página:

1. Modificar Valor

Asigna un nuevo valor a una propiedad existente:

```
alumno.edad = 21;
// Reemplaza el valor
```

2. Añadir Propiedad

Escribe una clave nueva y dale un valor:

```
alumno.email = "ana@web.es";
// Se crea al instante
```

3. Borrar con delete

Elimina la clave y su valor del objeto:

```
delete alumno.edad;
// Desaparece del objeto
```

💻 Ejemplo Completo Paso a Paso:

```
// 1. Creamos un objeto con datos iniciales
const producto = {
  id: 101,
  nombre: "Ratón Inalámbrico",
  precio: 15.50
};

// 2. Modificamos el precio (rebaja)
producto.precio = 12.99;

// 3. Añadimos una propiedad que no existía al principio
producto.stock = 25;
producto.disponible = true;

// 4. Eliminamos una propiedad con el operador delete
delete producto.id;

console.log(producto);
// Resultado en consola: { nombre: "Ratón Inalámbrico", precio: 12.99, stock: 25, disponible: true }
```

🔍 ¿Cómo saber si una propiedad existe en un objeto?

Existen dos métodos muy sencillos recomendados para principiantes:

El operador `in`

Devuelve `true` o `false` comprobando el nombre de la clave entre comillas:

```
if ("stock" in producto) {
  console.log("Tiene stock registrado");
}
```

Comprobar contra `undefined`

Si la propiedad no existe, devuelve `undefined`:

```
if (producto.descuento !== undefined) {
  // Tiene descuento
} else {
  console.log("Sin descuento");
}
```

> 💡 **¿Por qué usar 'delete' en vez de asignar 'null' o 'undefined'?**
> Si haces `producto.stock = null;`, la clave `stock` sigue existiendo dentro del objeto (solo que su valor es nulo). Con `delete producto.stock;` la propiedad es completamente eliminada de la memoria del objeto.

⚙️ 5. Métodos Sencillos y la Palabra Reservada 'this'

Hasta ahora los objetos guardaban sólo datos (textos, números...). Pero los objetos también pueden contener **acciones**: cuando una función se encuentra dentro de un objeto, se denomina **MÉTODO**.

¿Cómo se define un método?

En JavaScript moderno usamos la sintaxis abreviada directa:

```
const usuario = {
  nombre: "Marta",

  // Método del objeto:
  saludar() {
    return "¡Hola a todos!";
  }
};

// Se invoca con paréntesis ():
console.log(usuario.saludar());
```

¿Para qué sirve `this`?

`this` significa **"este mismo objeto"**. Permite al método leer o modificar sus propios datos internos:

```
const usuario = {
  nombre: "Marta",

  saludar() {
    // Accedemos a su propio nombre:
    return "Hola, soy " + this.nombre;
  }
};

console.log(usuario.saludar());
// "Hola, soy Marta"
```

🎯 Ejemplo Elemental: Un Coche que Acelera y Frena

Fíjate en cómo los métodos de este objeto modifican directamente el valor de `this.velocidad`:

```
const coche = {
  marca: "Toyota",
  velocidad: 0,

  acelerar() {
    this.velocidad += 10; // Sube 10 km/h
    console.log("Velocidad actual: " + this.velocidad + " km/h");
  },

  frenar() {
    if (this.velocidad > 0) {
      this.velocidad -= 10;
    }
    console.log("Velocidad actual: " + this.velocidad + " km/h");
  }
};

coche.acelerar(); // Velocidad actual: 10 km/h
coche.acelerar(); // Velocidad actual: 20 km/h
coche.frenar();   // Velocidad actual: 10 km/h
```

> ⚠️ **Regla de Oro para Principiantes: ¡Cuidado con las funciones flecha!**
> **NUNCA** definas métodos con función flecha `saludar: () => { ... }` si necesitas usar `this`. Las funciones flecha no tienen su propio `this` y apuntarán a la ventana global (`window`), provocando que `this.nombre` sea `undefined`. Usa siempre la sintaxis normal: `saludar() { ... }`.

📋 6. Arrays de Objetos: Colecciones del Mundo Real

En el desarrollo web profesional (tiendas online, listas de alumnos, noticias), los datos nunca vienen solos: vienen en un **Array donde cada elemento es un Objeto**. Esta es la estructura estándar de cualquier API o base de datos.

🛒 Catálogo de Ejemplo: Lista de Productos

```
const catalogo = [
  { id: 1, nombre: "Teclado", precio: 25, stock: 10 },
  { id: 2, nombre: "Ratón",   precio: 15, stock: 0 },
  { id: 3, nombre: "Monitor", precio: 180, stock: 5 }
];

// Acceso directo: primer producto del array
console.log(catalogo[0].nombre); // "Teclado"
console.log(catalogo[2].precio); // 180
```

🔁 ¿Cómo recorrer el array fácilmente?

Para principiantes, el bucle más limpio y comprensible es `for ... of`:

```
// Recorremos cada producto del catálogo:
for (const item of catalogo) {
  console.log("Producto: " + item.nombre + " | Precio: " + item.precio + "€");
}
```

🔎 Dos Operaciones Básicas Imprescindibles

1. Buscar un elemento (.find)

Devuelve el **primer objeto** que cumple la condición:

```
const encontrado = catalogo.find(p => p.id === 2);
console.log(encontrado.nombre); // "Ratón"
```

2. Filtrar elementos (.filter)

Devuelve un **nuevo array** con todos los que cumplen la condición:

```
// Solo los que tienen stock > 0
const disponibles = catalogo.filter(p => p.stock > 0);
console.log(disponibles.length); // 2
```

> 💡 **¿Cómo añadir un nuevo objeto al array?**
> Utiliza el método clásico de arrays `push()`: `catalogo.push({ id: 4, nombre: "Auriculares", precio: 30, stock: 8 });`. El nuevo objeto se añadirá automáticamente al final de la lista.

🌐 7. Mostrar Objetos en el DOM y Buenas Prácticas

El paso definitivo en el desarrollo web es conectar los datos del objeto con la pantalla del navegador (HTML). Hay dos formas principales de hacerlo:

Método 1: Campos individuales

Capturas cada etiqueta con `getElementById` y asignas su `textContent`:

```
const alumno = { nombre: "Lucía", nota: 9 };

document.getElementById("txtNombre").textContent = alumno.nombre;
document.getElementById("txtNota").textContent = alumno.nota;
```

Método 2: Plantilla con innerHTML

Generas la tarjeta completa usando plantillas de texto `` `...` ``:

```
const caja = document.getElementById("tarjeta");
caja.innerHTML = `
  <h3>\${alumno.nombre}</h3>
  <p>Calificación: \${alumno.nota}</p>
`;
```

⚠️ Los 4 Errores Más Frecuentes de Principiantes

ERROR TÍPICO	CÓDIGO ERRÓNEO	FORMA CORRECTA
1. Confundir `:` con `=`	`const p = { nombre = "Ana" };`	`const p = { nombre: "Ana" };`
2. Olvidar las comas	`const p = { a: 1  b: 2 };`	`const p = { a: 1, b: 2 };`
3. Olvidar el `this.` en métodos	`saludar() { return nombre; }`	`saludar() { return this.nombre; }`
4. Pintar el objeto directamente	`div.textContent = alumno; // [object Object]`	`div.textContent = alumno.nombre;`

🏆 **Resumen Clave del Ejercicio 19:**

1. Un objeto literal `{ clave: valor }` reúne datos relacionados de una misma entidad.
2. Accede a sus datos con el punto (`obj.prop`) o con corchetes si la clave viene de una variable (`obj[var]`).
3. Puedes añadir propiedades nuevas asignándoles valor (`obj.nueva = 10`) y borrarlas con `delete obj.prop`.
4. Dentro de un método, usa siempre `this.propiedad` para referirte a los datos de ese mismo objeto.
5. Para listas de elementos, utiliza un array de objetos `[{...}, {...}]` y recórrelo con `for...of`.